
Спецификация

Компонент Eco.String.C89

Статус: Черновик

Дата: Ноябрь 8, 2021

Версия: 1.0

Авторы	Компания
Владимир Башев	ПИРФ

Содержание

1. Обзор.....	3
1.1. Введение.....	3
1.2. Примечание.....	3
1.3. Ссылки.....	3
2. Компонент Eco.String.C89.....	4
2.1. IEcoStringC89 описание на ECO IDL.....	5
2.1.1. Функция memcpy.....	7
2.1.2. Функция memmove.....	7
2.1.3. Функция strcpy.....	7
2.1.4. Функция strncpy.....	7
2.1.5. Функция strcat.....	7
2.1.6. Функция strncat.....	7
2.1.7. Функция memcmp.....	7
2.1.8. Функция strcmp.....	8
2.1.9. Функция strxfrm.....	8
2.1.10. Функция memchr.....	8
2.1.11. Функция strchr.....	8
2.1.12. Функция strcspn.....	8
2.1.13. Функция strpbrk.....	8
2.1.14. Функция strrchr.....	9
2.1.15. Функция strspn.....	9
2.1.16. Функция strstr.....	9
2.1.17. Функция strtok.....	9
2.1.18. Функция memset.....	9
2.1.19. Функция strerror.....	9
2.1.20. Функция strlen.....	9
Приложение А: Обучающие программы.....	11

1. Обзор

Данный документ описывает требования к реализации компонента Eco.String.C89 (Компонент строка).

1.1. Введение

Описание.

1.2. Примечание

- Ключевые слова в документе

1.3. Ссылки

Данный параграф содержит ссылки на информацию, помогающую понять данный документ:

[] – наименование ссылки

Доступен по: <http://адрес>

2. Компонент Eco.String.C89

Компонент, имеет следующие описание:

2.1. IEcoStringC89 описание на ECO IDL

ECO IDL

```
import "IEcoBase1.h"

[
  object,
  uuid(00000000-0000-0000-0000-000073747231),
]

interface IEcoStringC89 : IEcoUnknown {

void*          memcpy          ([in] void* s1,
                                [in] void* s2,
                                [in] size_t n);

void*          memmove         ([in] void* s1,
                                [in] void* s2,
                                [in] size_t n);

char*          strcpy          ([in] char* s1,
                                [in] const char* s2);

char*          strncpy         ([in] char* s1,
                                [in] const char* s2,
                                [in] size_t n);

char*          strcat          ([in] char* s1,
                                [in] const char* s2);

char*          strncat         ([in] void* s1,
                                [in] void* s2,
                                [in] size_t n);

int            memcmp          ([in] const void* s1,
                                [in] const void* s2,
                                [in] size_t n);

int            strcmp          ([in] const char* s1,
                                [in] const char* s2);

int            strcoll         ([in] const char* s1,
                                [in] const char* s2);

int            strncmp         ([in] const char* s1,
                                [in] const char* s2,
                                [in] size_t n);

size_t         strxfrm         ([in] char* s1,
                                [in] const char* s2,
                                [in] size_t n);

void*          memchr          ([in] const void* s,
                                [in] int c,
                                [in] size_t n);

char*          strchr          ([in] const char* s,
                                [in] int n);

size_t         strcspn         ([in] const char* s1,
                                [in] const char* s2);

char*          strpbrk         ([in] const char* s1,
                                [in] const char* s2);
```

```
char*          strrchr          ([in] const char* s,  
                                  [in] int c);  
  
size_t         strspn          ([in] const char* s1,  
                                  [in] const char* s2);  
  
char*          strstr          ([in] const char* s1,  
                                  [in] const char* s2);  
  
char*          strtok          ([in] char* s1,  
                                  [in] const char* s2);  
  
void*          memset          ([in] void* s,  
                                  [in] int c,  
                                  [in] size_t n);  
  
char*          strerror        ([in] int errnum);  
  
size_t         strlen          ([in] const char* s);  
  
}
```

2.1.1. Функция memcpy

Библиотечная функция C копирует *n* символов из области памяти *src* в область памяти *dest*.

2.1.2. Функция memmove

Библиотечная функция C копирует *n* символов из *str2* в *str1*, но для перекрывающихся блоков памяти *memmove()* является более безопасным подходом, чем *memcpy()*.

2.1.3. Функция strcpy

Библиотечная функция C копирует строку, на которую указывает *src*, в *dest*.

2.1.4. Функция strncpy

Библиотечная функция C копирует до *n* символов из строки, на которую указывает *src*, в *dest*. В случае, когда длина *src* меньше длины *n*, остаток *dest* будет дополнен нулевыми байтами.

2.1.5. Функция strcat

Библиотечная функция C добавляет строку, на которую указывает *src*, в конец строки, на которую указывает *dest*.

2.1.6. Функция strncat

Библиотечная функция C добавляет строку, на которую указывает *src*, в конец строки, на которую указывает *dest* длиной до *n* символов.

2.1.7. Функция memchr

Библиотечная функция C сравнивает первые n байт области памяти str1 и области памяти str2.

2.1.8. Функция strcmp

Библиотечная функция C сравнивает строку, на которую указывает str1, со строкой, на которую указывает str2.

2.1.9. Функция strxfrm

Библиотечная функция C преобразует первые n символов строки src в текущую локаль и помещает их в строку dest.

2.1.10. Функция memchr

Библиотечная функция C ищет первое вхождение символа c (беззнакового символа) в первых n байтах строки, на которую указывает аргумент str.

2.1.11. Функция strchr

Библиотечная функция C ищет первое вхождение символа c (беззнакового char) в строке, на которую указывает аргумент str.

2.1.12. Функция strcspn

Библиотечная функция C использует время, указанное timer, для заполнения структуры tm значениями, представляющими соответствующее локальное время. Значение таймера разбивается в структуру tm и выражается в местном часовом поясе.

2.1.13. Функция strpbrk

Библиотечная функция C находит первый символ в строке str1, который соответствует любому символу, указанному в str2. Сюда не входят завершающие нулевые символы.

2.1.14. Функция strchr

Библиотечная функция C ищет последнее вхождение символа *c* (беззнакового *char*) в строке, на которую указывает аргумент *str*.

2.1.15. Функция strspn

Библиотечная функция C вычисляет длину начального сегмента *str1*, который полностью состоит из символов в *str2*.

2.1.16. Функция strstr

Библиотечная функция C использует время, указанное *timer*, для заполнения структуры *tm* значениями, представляющими соответствующее локальное время. Значение таймера разбивается в структуру *tm* и выражается в местном часовом поясе.

2.1.17. Функция strtok

Библиотечная функция C разбивает строку *str* на ряд лексем, используя разделитель *delim*.

2.1.18. Функция memset

Библиотечная функция C копирует символ *c* (беззнаковый символ) в первые *n* символов строки, на которую указывает аргумент *str*.

2.1.19. Функция strerror

Библиотечная функция C ищет во внутреннем массиве номер ошибки *errno* и возвращает указатель на строку сообщения об ошибке. Строки ошибок, создаваемые *strerror*, зависят от платформы разработки и компилятора.

2.1.20. Функция strlen

Библиотечная функция `C` вычисляет длину строки `str` до, но не включая завершающий символ `NULL`.

Приложение А: Обучающие программы

```

/*
 * <кодировка символов>
 *   Cyrillic (UTF-8 with signature) - Codepage 65001
 * </кодировка символов>
 *
 * <сводка>
 *   EcoStringC89
 * </сводка>
 *
 * <описание>
 *   Данный исходный файл является точкой входа
 * </описание>
 *
 * <автор>
 *   Copyright (c) 2018 Vladimir Bashev. All rights reserved.
 * </автор>
 *
 */

/* Eco OS */
#include "IEcoSystem1.h"
#include "IdEcoMemoryManager1.h"
#include "IdEcoInterfaceBus1.h"
#include "IdEcoStringC89.h"
#include "IdEcoStdIOC89.h"

/* Прототипы */
bool_t test_memcpy(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_memmove(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strcpy(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strncpy(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strcat(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strncat(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_memcmp(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strcmp(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strcoll(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strncmp(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strxfrm(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_memchr(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strchr(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strcspn(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strpbrk(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strchr(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strspn(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strstr(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strtok(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_memset(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strerror(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);
bool_t test_strlen(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO);

/*
 *
 */

```

```

* <сводка>
*   Функция EcoMain
* </сводка>
*
* <описание>
*   Функция EcoMain - точка входа
* </описание>
*
*/
int16_t EcoMain(IEcoUnknown* pIUnk) {
    int16_t result = -1;
    /* Указатель на системный интерфейс */
    IEcoSystem1* pISys = 0;
    /* Указатель на интерфейс работы с системной интерфейсной шиной */
    IEcoInterfaceBus1* pIBus = 0;
    /* Указатель на интерфейс работы с памятью */
    IEcoMemoryAllocator1* pIMem = 0;
    char_t* name = 0;
    char_t* copyName = 0;
    /* Указатель на тестируемый интерфейс */
    IEcoStringC89* pIStrng = 0;
    /* Указатель на интерфейс стандартного ввода/вывода */
    IEcoStdIOC89* pIStdIO = 0;

    /* Проверка и создание системного интрефейса */
    if (pISys == 0) {
        result = pIUnk->pVTbl->QueryInterface(pIUnk, &IID_IEcoSystem1, (void **)&pISys);
        if (result != 0 && pISys == 0) {
            /* Освобождение системного интерфейса в случае ошибки */
            goto Release;
        }
    }

    /* Получение интерфейса для работы с интерфейсной шиной */
    result = pISys->pVTbl->QueryInterface(pISys, &IID_IEcoInterfaceBus1, (void **)&pIBus);
    if (result != 0 || pIBus == 0) {
        /* Освобождение в случае ошибки */
        goto Release;
    }

#ifdef ECO_LIB
    /* Регистрация статического компонента для работы со списком */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoStringC89,
(IEcoUnknown*)GetIEcoComponentFactoryPtr_00000000000000000000000000000073747231);
    if (result != 0 ) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
    /* Регистрация статического компонента для работы со стандартным вводом/выводом */
    result = pIBus->pVTbl->RegisterComponent(pIBus, &CID_EcoStdIOC89,
(IEcoUnknown*)GetIEcoComponentFactoryPtr_00000000000000000000000000000053494F31);
    if (result != 0 ) {
        /* Освобождение в случае ошибки */
        goto Release;
    }
#endif
    /* Получение интерфейса управления памятью */
    result = pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoMemoryManager1, 0, &IID_IEcoMemoryAllocator1, (void **)&pIMem);
}

```

```

/* Проверка */
if (result != 0 && pIMem == 0) {
    /* Освобождение системного интерфейса в случае ошибки */
    goto Release;
}

/* Выделение блока памяти */
name = (char_t *)pIMem->pVTbl->Alloc(pIMem, 10);

/* Заполнение блока памяти */
pIMem->pVTbl->Fill(pIMem, name, 'a', 9);

/* Получение интерфейса со стандартным вводом/выводом */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoStdIOC89, 0, &IID_IEcoStdIOC89, (void**) &pIStdIO);
if (result != 0 && pIStdIO == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

/* Получение тестируемого интерфейса */
pIBus->pVTbl->QueryComponent(pIBus, &CID_EcoStringC89, 0, &IID_IEcoStringC89, (void**) &pIString);
if (result != 0 && pIString == 0) {
    /* Освобождение интерфейсов в случае ошибки */
    goto Release;
}

/* Тестирование методов интерфейса */
test_memcpy(pIString, pIStdIO);
test_memmove(pIString, pIStdIO);
test_strcpy(pIString, pIStdIO);
test_strncpy(pIString, pIStdIO);
test_strcat(pIString, pIStdIO);
test_strncat(pIString, pIStdIO);
test_memcmp(pIString, pIStdIO);
test_strcmp(pIString, pIStdIO);
test_strcoll(pIString, pIStdIO);
test_strncmp(pIString, pIStdIO);
test_strxfrm(pIString, pIStdIO);
test_memchr(pIString, pIStdIO);
test_strchr(pIString, pIStdIO);
test_strcspn(pIString, pIStdIO);
test_strpbrk(pIString, pIStdIO);
test_strrchr(pIString, pIStdIO);
test_strspn(pIString, pIStdIO);
test_strstr(pIString, pIStdIO);
test_strtok(pIString, pIStdIO);
test_memset(pIString, pIStdIO);
test_strerror(pIString, pIStdIO);
test_strlen(pIString, pIStdIO);

/* Освобождение блока памяти */
pIMem->pVTbl->Free(pIMem, name);

```

Release:

```

/* Освобождение интерфейса для работы с интерфейсной шиной */

```

```

if (pIBus != 0) {
    pIBus->pVTbl->Release(pIBus);
}

/* Освобождение интерфейса работы с памятью */
if (pIMem != 0) {
    pIMem->pVTbl->Release(pIMem);
}

/* Освобождение тестируемого интерфейса */
if (pIString != 0) {
    pIString->pVTbl->Release(pIString);
}

/* Освобождение интерфейса работы со стандартным вводом/выводом */
if (pIStdIO != 0) {
    pIStdIO->pVTbl->Release(pIStdIO);
}

/* Освобождение системного интерфейса */
if (pISys != 0) {
    pISys->pVTbl->Release(pISys);
}

return result;
}

bool_t test_memcpy(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO) {
    const char src[50] = "https://www.peerf.me";
    char dest[50];

    pIString->pVTbl->strcpy(pIString, dest, "Hellooo!!");
    pIStdIO->pVTbl->printf(pIStdIO, "Before memcpy dest = %s\n", dest);
    pIString->pVTbl->memcpy(pIString, dest, src, pIString->pVTbl->strlen(pIString, src)+1);
    pIStdIO->pVTbl->printf(pIStdIO, "After memcpy dest = %s\n", dest);

    return 1;
}

bool_t test_memmove(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO) {
    char dest[] = "oldstring";
    const char src[] = "newstring";

    pIStdIO->pVTbl->printf(pIStdIO, "Before memmove dest = %s, src = %s\n", dest, src);
    pIString->pVTbl->memmove(pIString, dest, src, 9);
    pIStdIO->pVTbl->printf(pIStdIO, "After memmove dest = %s, src = %s\n", dest, src);
    return 1;
}

bool_t test_strcpy(IEcoStringC89* pIString, IEcoStdIOC89* pIStdIO) {
    char src[40];
    char dest[100];

    pIString->pVTbl->memset(pIString, dest, '\0', sizeof(dest));
    pIString->pVTbl->strcpy(pIString, src, "This is peerf.me");
    pIString->pVTbl->strcpy(pIString, dest, src);

    pIStdIO->pVTbl->printf(pIStdIO, "Final copied string : %s\n", dest);
}

```

```

    return 1;
}

bool_t test_strncpy(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    char src[40];
    char dest[12];

    pIString->pVTbl->memset(pIString, dest, '\0', sizeof(dest));
    pIString->pVTbl->strcpy(pIString, src, "This is tpeerf.me");
    pIString->pVTbl->strncpy(pIString, dest, src, 10);

    pIStdIO->pVTbl->printf(pIStdIO, "Final copied string : %s\n", dest);

    return 1;
}

bool_t test_strcat(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    char src[50], dest[50];

    pIString->pVTbl->strcpy(pIString, src, "This is source");
    pIString->pVTbl->strcpy(pIString, dest, "This is destination");

    pIString->pVTbl->strcat(pIString, dest, src);

    pIStdIO->pVTbl->printf(pIStdIO, "Final destination string : |%s|\n", dest);
    return 1;
}

bool_t test_strncat(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    char src[50], dest[50];

    pIString->pVTbl->strcpy(pIString, src, "This is source");
    pIString->pVTbl->strcpy(pIString, dest, "This is destination");

    pIString->pVTbl->strncat(pIString, dest, src, 15);

    pIStdIO->pVTbl->printf(pIStdIO, "Final destination string : |%s|\n", dest);

    return 1;
}

bool_t test_memcmp(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    char str1[15];
    char str2[15];
    int ret;

    pIString->pVTbl->memcpy(pIString, str1, "abcdef", 6);
    pIString->pVTbl->memcpy(pIString, str2, "ABCDEF", 6);

    ret = pIString->pVTbl->memcmp(pIString, str1, str2, 5);

    if(ret > 0) {
        pIStdIO->pVTbl->printf(pIStdIO, "str2 is less than str1\n");
    } else if(ret < 0) {
        pIStdIO->pVTbl->printf(pIStdIO, "str1 is less than str2\n");
    } else {
        pIStdIO->pVTbl->printf(pIStdIO, "str1 is equal to str2\n");
    }
}

```

```

    return 1;
}

bool_t test_strcmp(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    char str1[15];
    char str2[15];
    int ret;

    pIString->pVTbl->strcpy(pIString, str1, "abcdef");
    pIString->pVTbl->strcpy(pIString, str2, "ABCDEF");

    ret = pIString->pVTbl->strcmp(pIString, str1, str2);

    if(ret < 0) {
        pIStdIO->pVTbl->printf(pIStdIO, "str1 is less than str2\n");
    } else if(ret > 0) {
        pIStdIO->pVTbl->printf(pIStdIO, "str2 is less than str1\n");
    } else {
        pIStdIO->pVTbl->printf(pIStdIO, "str1 is equal to str2\n");
    }
    return 1;
}

bool_t test_strcoll(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    char str1[15];
    char str2[15];
    int ret;

    pIString->pVTbl->strcpy(pIString, str1, "abc");
    pIString->pVTbl->strcpy(pIString, str2, "ABC");

    ret = pIString->pVTbl->strcoll(pIString, str1, str2);

    if(ret > 0) {
        pIStdIO->pVTbl->printf(pIStdIO, "str1 is less than str2\n");
    } else if(ret < 0) {
        pIStdIO->pVTbl->printf(pIStdIO, "str2 is less than str1\n");
    } else {
        pIStdIO->pVTbl->printf(pIStdIO, "str1 is equal to str2\n");
    }
    return 1;
}

bool_t test_strncmp(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    char str1[15];
    char str2[15];
    int ret;

    pIString->pVTbl->strcpy(pIString, str1, "abcdef");
    pIString->pVTbl->strcpy(pIString, str2, "ABCDEF");

    ret = pIString->pVTbl->strncmp(pIString, str1, str2, 4);

    if(ret < 0) {
        pIStdIO->pVTbl->printf(pIStdIO, "str1 is less than str2\n");
    } else if(ret > 0) {
        pIStdIO->pVTbl->printf(pIStdIO, "str2 is less than str1\n");
    }
}

```



```

    } else {
        pIStdIO->pVTbl->printf(pIStdIO, "str1 is equal to str2\n");
    }

    return 1;
}

bool_t test_strxfrm(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    char dest[20];
    char src[20];
    int len;

    pIString->pVTbl->strcpy(pIString, src, "Tutorials ");
    len = pIString->pVTbl->strxfrm(pIString, dest, src, 20);

    pIStdIO->pVTbl->printf(pIStdIO, "Length of string |%s| is: %d\n", dest, len);

    return 1;
}

bool_t test_memchr(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    const char str[] = "https://www.peerf.me";
    const char ch = '.';
    char *ret;

    ret = pIString->pVTbl->memchr(pIString, str, ch, pIString->pVTbl->strlen(pIString, str));

    pIStdIO->pVTbl->printf(pIStdIO, "String after |%c| is - |%s|\n", ch, ret);

    return 1;
}

bool_t test_strchr(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    const char str[] = "https://www.peerf.me";
    const char ch = '.';
    char *ret;

    ret = pIString->pVTbl->strchr(pIString, str, ch);

    pIStdIO->pVTbl->printf(pIStdIO, "String after |%c| is - |%s|\n", ch, ret);

    return 1;
}

bool_t test_strcspn(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    int len;
    const char str1[] = "ABCDEF4960910";
    const char str2[] = "013";

    len = pIString->pVTbl->strcspn(pIString, str1, str2);

    pIStdIO->pVTbl->printf(pIStdIO, "First matched character is at %d\n", len + 1);
    return 1;
}

bool_t test_strpbrk(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    const char str1[] = "abcde2fghi3jk4l";
    const char str2[] = "34";

```

```

char *ret;

ret = pIString->pVTbl->strpbrk(pIString, str1, str2);
if(ret) {
    pIStdIO->pVTbl->printf(pIStdIO, "First matching character: %c\n", *ret);
} else {
    pIStdIO->pVTbl->printf(pIStdIO, "Character not found\n");
}
return 1;
}

bool_t test_strrchr(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    const char str[] = "https://www.peerf.me";
    const char ch = '.';
    char *ret;

    ret = pIString->pVTbl->strrchr(pIString, str, ch);

    pIStdIO->pVTbl->printf(pIStdIO, "String after %c is - %s\n", ch, ret);

    return 1;
}

bool_t test_strspn(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    int len;
    const char str1[] = "ABCDEFGH019874";
    const char str2[] = "ABCD";

    len = pIString->pVTbl->strspn(pIString, str1, str2);

    pIStdIO->pVTbl->printf(pIStdIO, "Length of initial segment matching %d\n", len );
    return 1;
}

bool_t test_strstr(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    const char haystack[20] = "TutorialsPoint";
    const char needle[10] = "Point";
    char *ret;

    ret = pIString->pVTbl->strstr(pIString, haystack, needle);

    pIStdIO->pVTbl->printf(pIStdIO, "The substring is: %s\n", ret);
    return 1;
}

bool_t test_strtok(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    char str[80] = "This is - www.peerf.me - website";
    const char s[2] = "-";
    char *token;

    /* get the first token */
    token = pIString->pVTbl->strtok(pIString, str, s);

    /* walk through other tokens */
    while( token != NULL ) {
        pIStdIO->pVTbl->printf(pIStdIO, " %s\n", token );

        token = pIString->pVTbl->strtok(pIString, NULL, s);
    }
}

```

```

    }
    return 1;
}

bool_t test_memset(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    char str[50];

    pIString->pVTbl->strcpy(pIString, str, "This is string.h library function");
    puts(str);

    pIString->pVTbl->memset(pIString, str, '$', 7);
    pIStdIO->pVTbl->puts(pIStdIO, str);
    return 1;
}

bool_t test_strerror(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    FILE *fp;

    fp = pIStdIO->pVTbl->fopen(pIStdIO, "file.txt", "r");
    if( fp == NULL ) {
        pIStdIO->pVTbl->printf(pIStdIO, "Error: %s\n", pIString->pVTbl->strerror(pIString, errno));
    }
    return 1;
}

bool_t test_strlen(IEcoStringC89* pIString, IEcoStdIOc89* pIStdIO) {
    char str[50];
    int len;

    pIString->pVTbl->strcpy(pIString, str, "This is peerf.me");

    len = pIString->pVTbl->strlen(pIString, str);
    pIStdIO->pVTbl->printf(pIStdIO, "Length of |%s| is |%d|\n", str, len);
    return 1;
}

```